École Nationale des Sciences Appliquées – Tétouan
Année Académique 2024-2025

Rapport De Travaux Pratiques

Algorithmes Value Iteration et Policy Iteration

"Environnement CliffWalking"

Etudiants: Abdessamad Ait brahim

Encadrant: Anas Belcaid

Module: Deep Learning

Filière: Big Data and AI

Date: December 2025

---

1 Introduction

Le rapport présente une étude comparative de deux algorithmes fondamentaux dans le domaine de la programmation dynamique, à savoir :**Value itération** et **Policy itération**. Ces deux derniers sont appliqués à l'environnement **CliffWalking-v1** de **Gymnasium** qui constitue un problème classique d'apprentissage par renforcement.

Les objectifs de ce TP sont :

- Implementataion des deux algorithmes .
- Analyse des performances à savoir: **La convergence, l'effecacité, et la qualité des solutions)**.
- Visualisation des politiques optimales obtenues.
- Compréhension des différences théoriques et pratiques entre ces approches .

2 Présentation de l'environnement

CliffWalking est un environnement constitue de une grille de **4 lignes et 12 colonnes**, où un agent doit naviguer depuis le point de **départ S** (start) , plus précisément l'état 36 vers le **but G** (goal) correspondant à l'état 47 , tout en évitant une falaise mortelle

Les caractéristiques:

- Nombres d'états : **48 (4x12)**.
- Nombres d'actions: **4 (↑, →, ↓, ←)**.
- Point du départ : **Etat 36 (coin inférieur à gauche)**.
- Point d'arrivé : **Etat 47 (coin inférieur à droite)**.
- Zone dangereuses : **Etat 37 à 46**.

Structure de récompenses:

- **-1**: déplacement normal.
- **-100**: tomber dans la falise –> retour au départ.
- **0**: atteindre le but .

Dynamique déterministe : Chaque action produit le même resultat, ce qui simplifier l'application des algorithmes .

3 Fondements mathématiques

3.1 Processus de décision Markovien (MDP)

Un MDP est définit par un tuple (S, A, P, R, γ)

- **S**: Ensemble des états.
- **A**: Ensemble des actions.
- **R** : Fonction de récompense, qui associe une récompense à la transition de l'état s vers l'état s' en exécutant l'action a, notée $R(s, a, s')$.
- $\boldsymbol{\gamma}$: facteur d'escompte, qui pondère l'importance des récompenses futures.

3.2 Fonction de valeur V(s) et l'equation de Bellman

Cette fonction représente l'utilité espérée en partant de l'état **s**:

$$V(s) = \max_a \sum_{s'} P(s' \mid s, a) \left[R(s, a, s') + \gamma V(s') \right] \tag{1}$$

NB: L'équation est au coeur des deux algorithmes étudiés.

3.3 Politique optimale

Une politique $\pi : S \to A$ associe à chaque état une action.La politique $\pi*$ a le but de maximiser la fonction de valeur

$$\pi^*(s) = \arg\max_a \sum_{s'} P(s' \mid s, a) \left[R(s, a, s') + \gamma V^*(s') \right] \tag{2}$$

4 Méthodologie

4.1 Value Iteration

Cet algorithme sert à calculer itérativement la fonction de valeur **V***, puis en déduit la politique optimale.

4.1.1 Processus algorithmique

Le processus algorithmique peut être résumé comme suit :

1. **Initialisation :**
$$V(s) = 0 \quad \forall s$$
2. **Itération :** Pour chaque état s, mise à jour synchrone :
$$V(s) \leftarrow \max_a \sum_{s'} P(s' \mid s, a) \left[R(s, a, s') + \gamma V(s') \right]$$

3. **Convergence :** Arrêt lorsque :

$$|V_{\text{new}}(s) - V_{\text{old}}(s)| < \theta \quad \forall s$$

4. **Extraction de la politique :**

$$\pi(s) = \arg\max_a \sum_{s'} P(s' \mid s, a) \left[R(s, a, s') + \gamma V(s')\right]$$

Paramètres utilisés :

- Facteur d'escompte : $\gamma = 0.9$
- Seuil de convergence : $\theta = 10^{-6}$
- Itérations maximales : 5000

4.2 Policy Itération

Cet algorithme alterne entre deux phases: **L'évaluation de la politique courante et son amélioration**

4.2.1 Processus algorithmique

Le processus algorithmique de la **Policy Iteration** se déroule en deux phases principales:

1. **Phase 1 : Évaluation de la politique** Pour chaque état s, on calcule la valeur de la politique actuelle π :

$$V^{\pi}(s) = \sum_{s'} P(s' \mid s, \pi(s)) \left[R(s, \pi(s), s') + \gamma V^{\pi}(s')\right]$$

2. **Phase 2 : Amélioration de la politique** La politique est mise à jour en choisissant l'action qui maximise la valeur attendue :

$$\pi_{\text{new}}(s) = \arg\max_a \sum_{s'} P(s' \mid s, a) \left[R(s, a, s') + \gamma V^{\pi}(s')\right]$$

3. **Critère d'arrêt :** La politique ne change plus, ce qui indique que la stabilité est atteinte.

Paramètres utilisés :

- Facteur d'escompte : $\gamma = 0.9$
- Seuil d'évaluation : $\theta = 10^{-6}$
- Itérations maximales : 100

5 Résultats et Analyses

5.1 Convergence des algorithmes

La convergence de ces deux algorithmes est rapide vers une solution optimale :

- **Value Itération** : environ 15 itérations
- **Policy Itération** : environ 15 itérations

Temps d'exécution :

- **Value Itération** : environ 0.0094 s
- **Policy Itération** : environ 0.3259 s

Value itération est 35 fois plus rapide, car il fait des mises à jour simples, tandis que Policy itération doit évaluer complètement chaque politique à chaque itération.

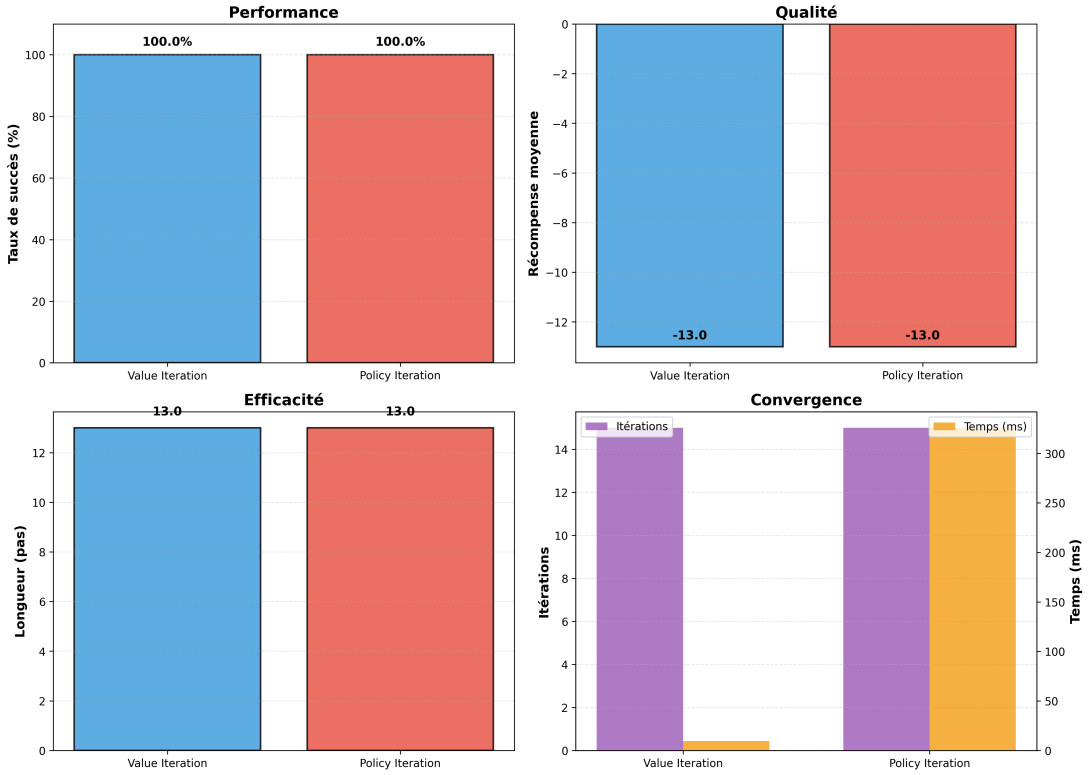


Figure 1: Comparaison complète .

Le graphique ci-dessus (Figure 1) montre que les résultats sont similaires pour les deux algorithmes.

- **Taux de succès** : 100%
- **Récompense moyenne** : -13.00
- **Longueur moyenne** : 13.00

Ces résultats confirment que la convergence vers **la même politique optimale** est parfaite au niveau de la performance.

5.2 Politiques optimales

Les politiques obtenues sont identiques:

```
[→] [→] [→] [→] [→] [→] [→] [→] [→] [→] [→] [↓]
[→] [→] [→] [→] [→] [→] [→] [→] [→] [→] [→] [↓]
[→] [→] [→] [→] [→] [→] [→] [→] [→] [→] [→] [↓]
[S] [C] [C] [C] [C] [C] [C] [C] [C] [C] [C] [G]
```

Stratégie optimale : Contournement par le haut en 13 étapes.

- **Montée 3 cases pour eviter la falaise**
- **Traversée horizontale 11 cases**
- **Descente 3 cases vers le but**

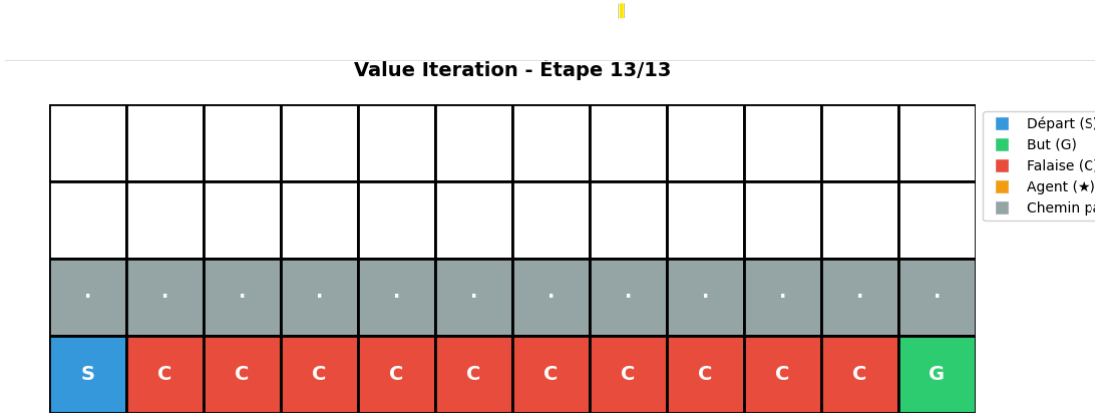


Figure 2: Animation de la trajectoire optimale (Value iteration)

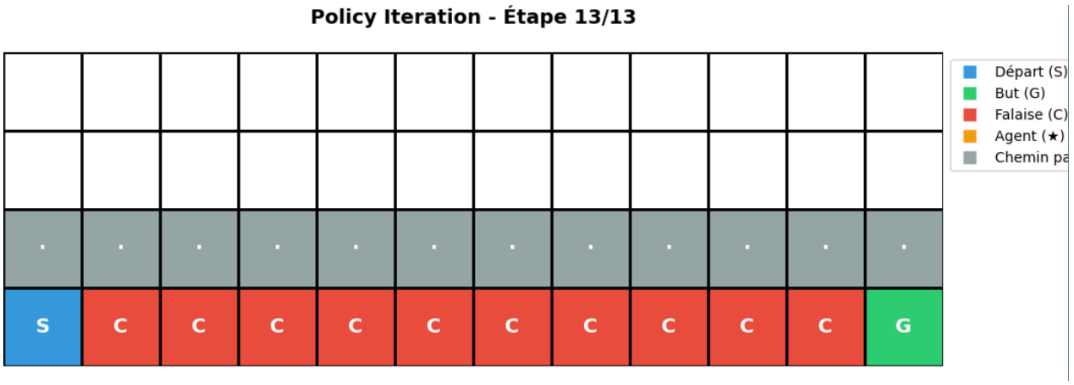


Figure 3: Animation de la trajectoire optimale (Policy iteration)

Les deux GIFs animeés sont identiques en montrant 13 etapes du parcours optimal.

6 Conclusion

Durant ce TP, les deux algorithmes produisent exactement même parcours optimal en 13 etapes , mais **Value Iteration** est 35 fois plus rapide , ce qui affirme le choix optimal pour cet environnement.